

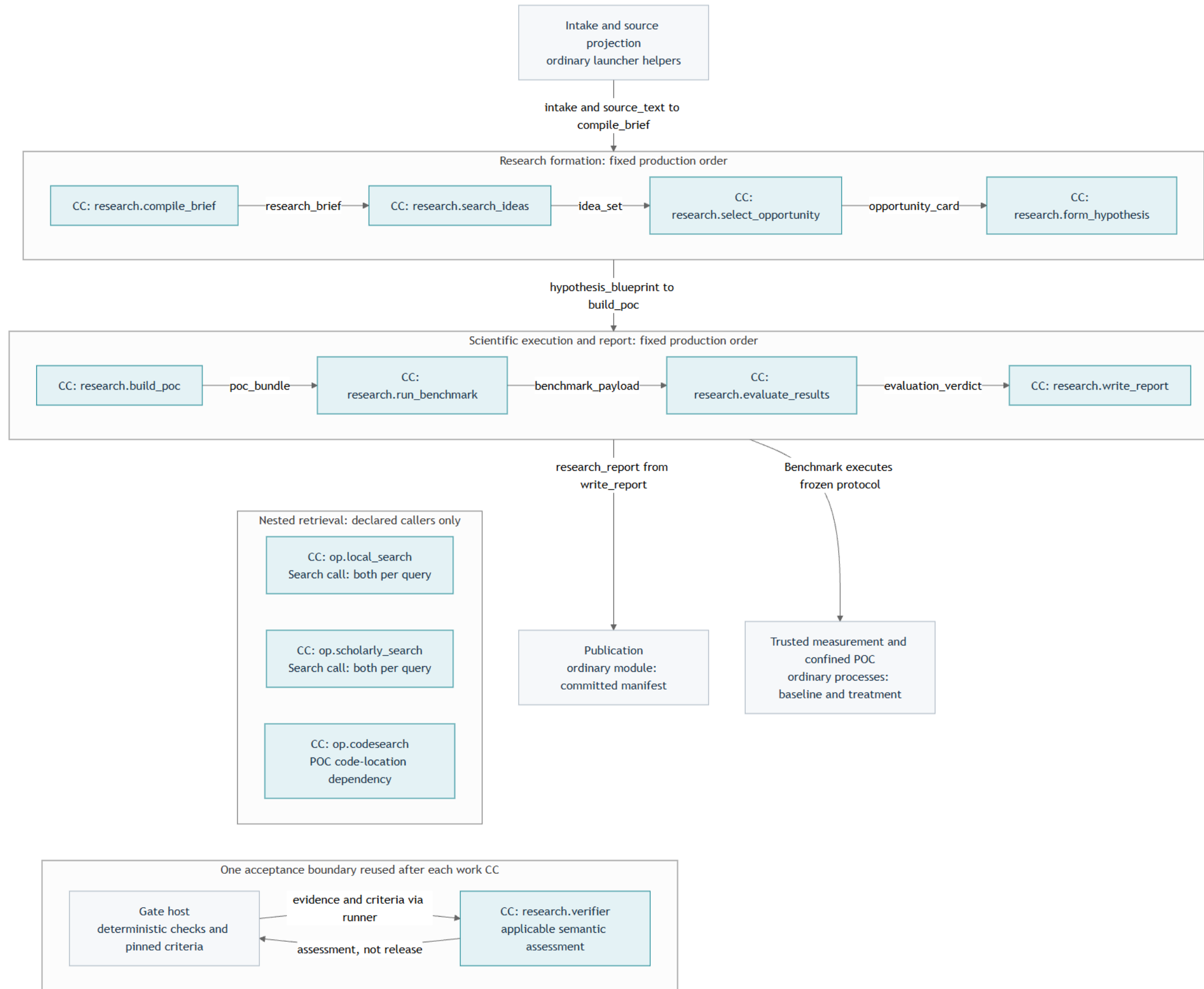
01 | What the architecture looks like - and why

5 October 2026 | Current architecture draft | The decisions and diagrams are design evidence, not proof of an implemented system.

- **One local Docker application:** modular monolith with internal protected processes. One deployment keeps operations simple; runner, model bridge, generated-code execution and private oracle keep distinct trust boundaries. **Pattern:** [Docker restricted containers](#).
- **Twelve CC identities:** eight research work CCs, one shared verifier and three nested retrieval CCs. Production runs Brief -> Search -> Screening -> Hypothesis -> POC -> scientific Benchmark -> Evaluation -> Report, then ordinary publication. **Pattern:** [Kubeflow typed components](#).
- **Clear authority:** supervisor chooses when to dispatch; capsules perform their capability; Gate host folds pinned checks/criteria. A committed advancing Verification and release must exist before the next node. **Patterns:** [OPA profiles](#) and [Temporal durable progress](#).
- **Codex through an adapter:** protected bridge, separate persistent login, serialized credential owner, captured calls and bounded reservations. Generated code never receives credentials. **Pattern:** [Codex persistent authentication](#); replace provider/auth behind stable interfaces.
- **Separate tracks:** fixed production; required isolated offline RSI with private oracle; optional whitelisted planner/routing/compiler/Code Mode experiments. Planner is not a CC and never changes production. Candidate admission, assurance and human activation are separate. **Pattern:** [MLflow versions and aliases](#).
- **Controlled input/output:** local resources are validated and snapshotted; literature enters through bounded search; trusted measurements ground scientific results. Reports and sealed evidence leave through authorized manifests and benchmark HTTP. **Patterns:** [HTTP resource semantics](#) and [OpenAPI](#).
- **Read the next diagrams:** blue boxes identify every CC; dashed amber boxes mark conditional track/session participation. They do not make production stages optional. Search calls both local and scholarly retrieval per query; CodeSearch supports POC. Required capture is summarized, not drawn from every capsule.

Every capsule in the research path

05 OCT 2026 / MERMAID SOURCE VIEW / DRAFT, NOT RUNTIME ACCEPTANCE

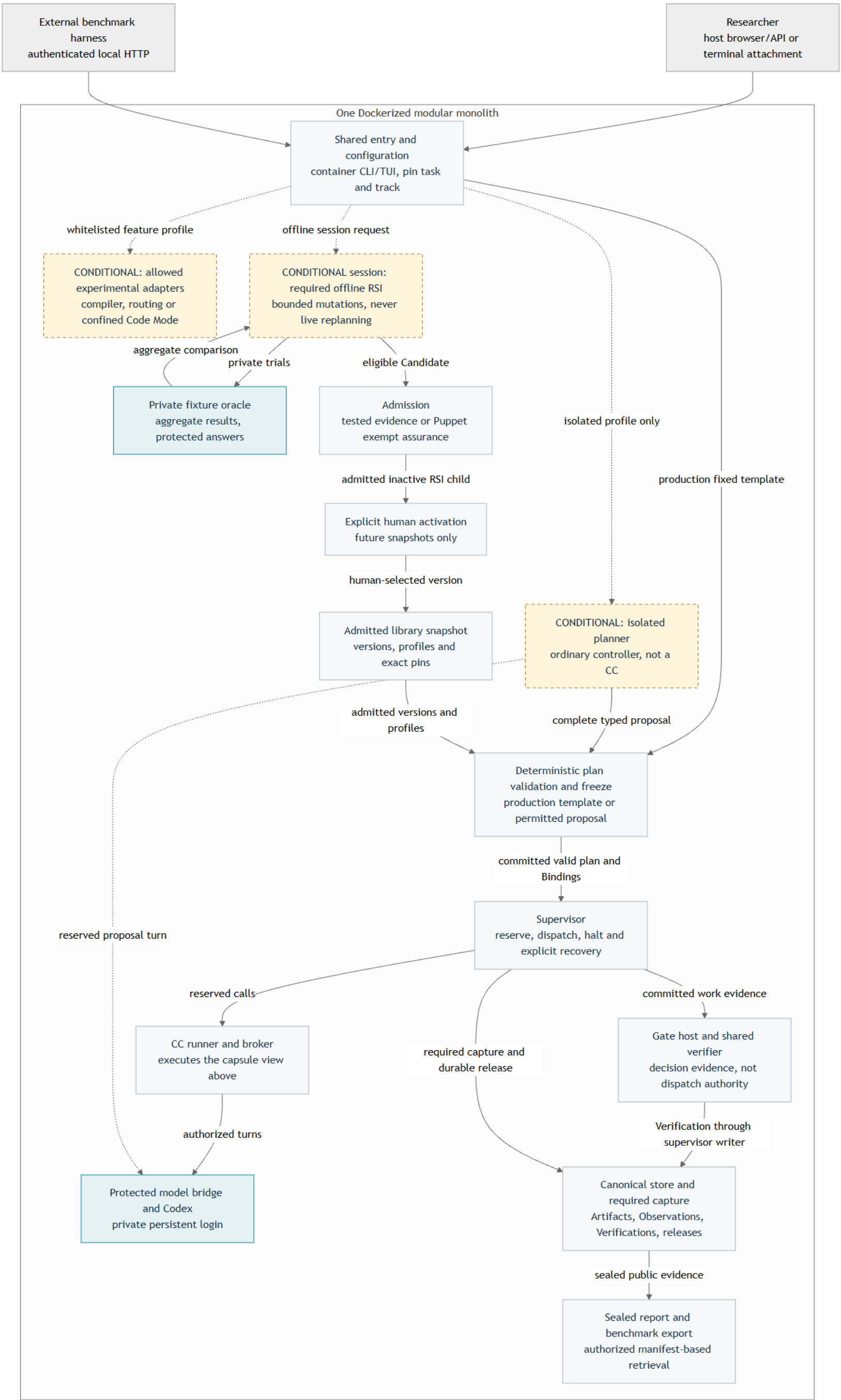


Eight fixed work CCs; three nested retrieval CCs; one shared verifier. Supporting insets are not later stages.
Search calls BOTH local and scholarly operators per query; POC calls CodeSearch. Gates apply after every work CC.

Editable Mermaid and canonical owner links: <system/overall-draft.md>

One application, conditional tracks and authority

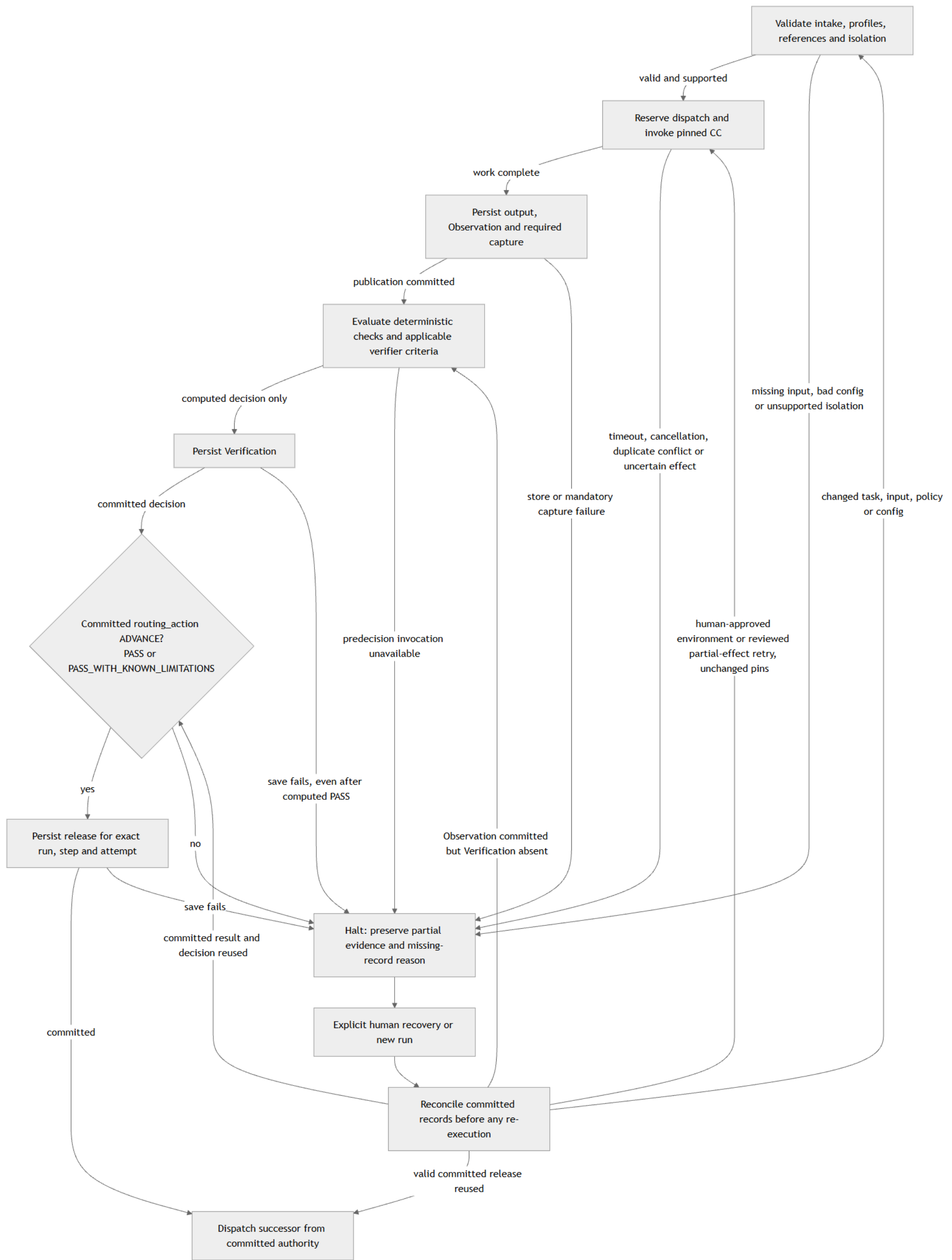
05 OCT 2026 / MERMAID SOURCE VIEW / DRAFT, NOT RUNTIME ACCEPTANCE



Dashed amber boxes are conditional track/session participation. Required offline RSI is not in every production run. Compiler before proposal; routing inside model calls; Code Mode in an isolated worker. Their feature box is an annotation.

Failure and recovery: what permits the next step?

05 OCT 2026 / MERMAID SOURCE VIEW / DRAFT, NOT RUNTIME ACCEPTANCE



Advancement requires committed PASS or PASS_WITH_KNOWN_LIMITATIONS and routing_action ADVANCE, then durable release. Ordinary failed checks produce Verification. Explicit recovery reuses committed evidence and never silently repeats uncertain work.

Editable Mermaid and canonical owner links: [system/information-flow.md](#)

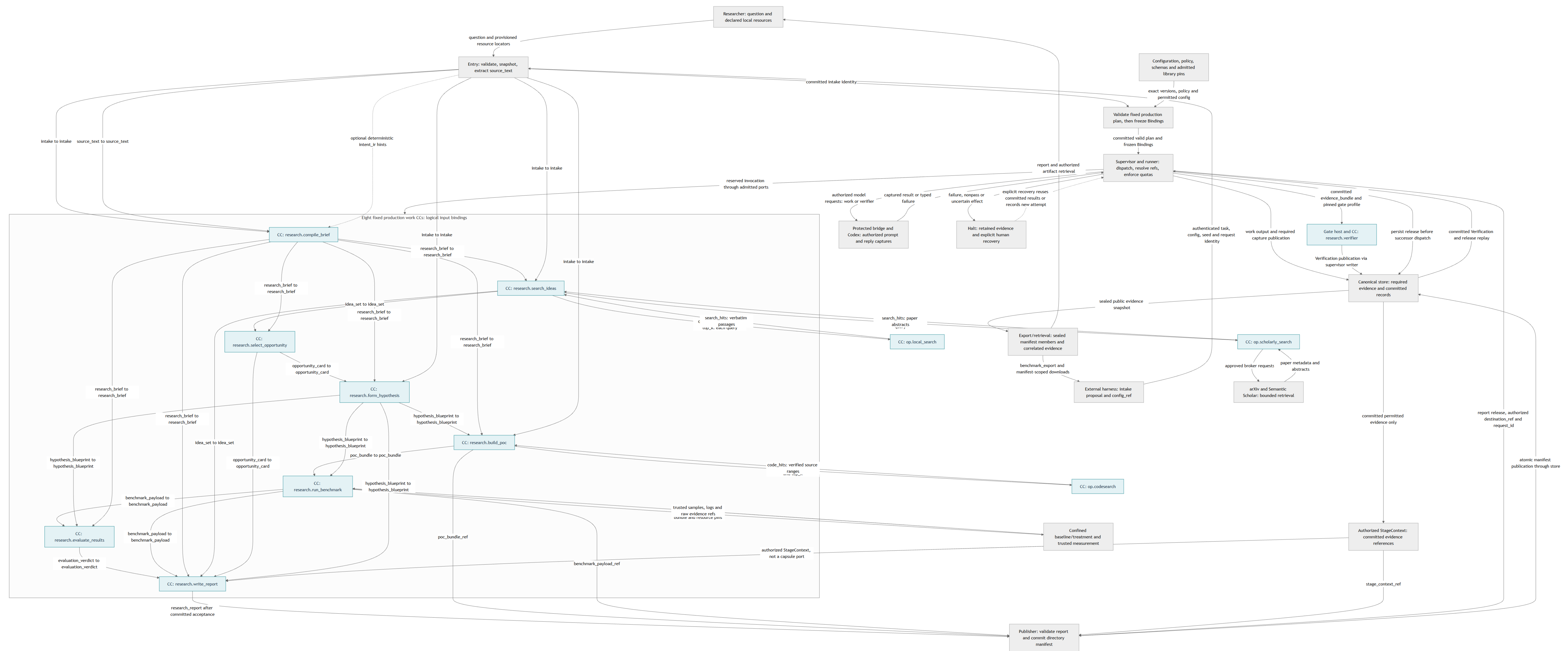
05 | Depth, testing policy and coding value

- **Depth without clutter:** the overview shows responsibilities and placement; the next maps show all capsules and conditional paths; failure/recovery shows temporal authority. The final zoom appendix contains every required production input binding and publisher fan-in. Full, observability-hidden, RSI-hidden and core Mermaid variants are linked from the sharing index.
- **One owning contract:** schemas, APIs, config keys, records and errors have canonical definitions. Capsules pin compatible versions; the plan binds producers to consumers. Coders can build separate modules against these boundaries instead of deciding product behavior.
- **Checks before joining:** schema/example/link/graph checks; blind producer/consumer derivations; seam canaries; fake adjacent services for independent invocation. **Pattern:** [Pact consumer/provider verification](#), adapted to local design checks.
- **Adversarial cases:** missing/wrong-version inputs, duplicates, timeout/cancel, corrupt refs, lost Gate/release writes, interrupted publication, denied paths/network/credentials, planner rejection and RSI budget reset. Observe actual records and next-node counts, not only PASS text.
- **Connected acceptance:** production through report; halt/recovery; offline RSI through admission/manual activation; isolated planner through valid dispatch; benchmark invocation/export. Scientific FAIL can still be a valid completed report; invalid evidence cannot.
- **Development policy:** freeze sources; change owners and affected consumers/diagrams/examples together; invalidate affected evidence; keep released interfaces immutable or migrate explicitly. Small source-verified AI reviews precede human decision briefs. **Pattern:** [Google small-change review](#).
- **Honest readiness:** architecture lint, contract and story checks pass. Docker isolation/platforms, crash durability, native integration, real Codex behavior and trusted measurements still need implementation evidence. Coders create Spec Kit and execute tests; architecture defines the required observations.

Canonical reading hub: [showcase/README.md](#). The full report and exact contracts remain linked there; this presentation is a dated derived view.

Detailed core: actual information bindings

05 OCT 2026 / MERMAID SOURCE VIEW / DRAFT, NOT RUNTIME ACCEPTANCE



Zoom map: all 23 required production input bindings plus nested calls, publisher fan-in, I/O and durable authority.
Derived observability and offline RSI branches are hidden here, not disabled. Full and other filters are linked from the sharing index.

Editable Mermaid and canonical owner links: [system/information-flow.md](#)